When Graph Neural Network Accuracy Misleads: Evaluation Shortcuts and Topology Counterfactuals in ICS/SCADA Intrusion Detection

Misbah Shaheen

School of Electrical Engineering and Computer Science, National University of Sciences and Technology (NUST), Islamabad, Pakistan

Abstract

Graph neural networks (GNNs) are widely proposed for intrusion detection in industrial control systems (ICS), but reported performance depends on how the data are split and on whether the graph contributes anything beyond packet content. We audit both questions on SCADANet (534,841 flows, 13 attack classes) and BATADAL (693 sliding windows) with a GraphSAGE edge classifier, non-graph and alternative-GNN baselines, topology counterfactuals and a strict causal temporal protocol, using ten seeds for the main experiments. The split changes what is measured: an IP-held-out split (Track A) contains 2 of 13 attack classes, and a flow-level split (Track B) shares 99.6% of IP pairs between training and test data. On Track B, GraphSAGE (F1 0.985) is practically indistinguishable from an MLP (0.984), XGBoost (0.986) and a GCN (0.986), and random or degree-preserving rewiring of the topology does not lower F1 (0.985 in both cases). Under strict chronological evaluation, tuned GraphSAGE, MLP and XGBoost fall by about 0.30 F1 (to 0.68–0.69); in these configurations the missed recall is largely accounted for by one class, `vuln_scan`, which makes up 45% of test-period attack flows, although other configurations miss further classes. A GCN (F1 0.914) and a topology-only model (0.915) detect `vuln_scan`, but the topology-only model misses three other attack classes entirely. We do not have an explanation for these two results: topology-only rewiring controls were not run, so relational learning cannot be distinguished from degree, hub-layout or novelty shortcuts. Threshold calibration, rather than per-host feature normalization, substantially reduces false positives on one of the two dominant hosts (FPR 0.277 to 0.144 on Track B), and the calibrated threshold also lowers `os_scan` recall from 0.997 to about 0.50. The study is limited to two simulated or testbed-derived datasets and does not address attack-path prediction or physical-impact modeling.

Keywords: graph neural networks, shortcut learning, industrial control systems, intrusion detection, evaluation methodology, temporal generalization, SCADA

1. Introduction

ICS and SCADA networks are both communication networks and physical processes: a malicious flow can become a manipulated sensor reading or an unsafe actuator command (Taormina et al., 2018). Graph representations are therefore a natural fit for intrusion detection in this setting. Nodes and edges carry packet-level or sensor-level content, while the topology records which devices can influence which others. A growing body of work applies GraphSAGE-family and other GNN architectures to flow classification and reports strong aggregate metrics (Lo et al., 2022; Sun et al., 2024; Zhong et al., 2024).

Strong aggregate metrics do not establish that the graph is doing the work. Two problems recur. First, the train/test split determines what the test set contains: a split by IP address, by flow, or by time evaluates different abilities, and some splits leave entire attack classes out. Second, a GNN may reach its score from packet content, node identity or degree rather than from relational structure, which is the situation described by the shortcut-learning literature (Geirhos et al., 2020). If a content-only classifier matches the GNN, the graph machinery adds cost without adding detection capability.

This paper examines both problems empirically on two ICS datasets and organizes the evidence around

Email address: `mshaheen.bsds23seecs@seecs.edu.pk` (Misbah Shaheen)

three research questions.

RQ1 How strongly do train/test partitioning strategies, and the leakage or shortcut conditions they introduce, affect reported GNN-based ICS intrusion-detection performance?

RQ2 Under leakage-safe evaluation, how much predictive value comes from genuine graph topology compared with packet or flow content and shortcut features?

RQ3 Do the observed effects remain consistent across time, datasets, attack types, model families and random seeds?

The contributions are:

1. A split audit of six SCADANet and BATADAL evaluation protocols that quantifies attack-class coverage, host and IP-pair overlap, and temporal overlap, together with a strict causal temporal protocol (SCADANet) and a purged chronological split (BATADAL) designed to eliminate identified temporal-overlap and future-topology leakage in the earlier protocols.
2. A comparison of logistic regression, an MLP, XGBoost, GCN, GAT and GraphSAGE under identical splits, features, seeds and validation-only threshold selection, with six topology and content counterfactuals (conditions A–F).
3. An evaluation of per-host feature normalization against threshold calibration for host-concentrated false positives (conditions H1–H4), including the effect on per-attack recall.
4. Ten-seed estimates with 95% confidence intervals, paired effect sizes, calibration and an intervention test of feature attributions. Negative and unresolved results are reported as found.

2. Related Work

GNN-based intrusion detection.. Graph-based detectors treat hosts or flows as graph elements and use message passing to combine topology with content. E-GraphSAGE (Lo et al., 2022) and GNN-IDS (Sun et al., 2024) apply GraphSAGE-style models to network-flow classification, and surveys (Zhong et al., 2024) identify evaluation design, in particular what is held out between training and test, as a persistent weakness of the field. Other work targets the cyber-physical setting directly: Sweeten et al. (2023) fuse network and sensor features in a heterogeneous GNN for smart grids, and Li and Chasaki (2024) add non-adjacent "express edges" to capture multi-hop propagation. These studies report aggregate detection metrics but do not test whether the graph structure, rather than flow content or the split, produces them. We do not propose an architecture; we test whether an existing one earns its complexity.

ICS and SCADA graph detection.. BATADAL (Taormina et al., 2018) provides simulated water-distribution SCADA readings and is mostly used with time-series and autoencoder methods (Somma, 2025; Conti et al., 2021). Spatio-temporal GNNs have been evaluated on the SWaT testbed, where Koistinen et al. (2026) caution that maximizing F1 can select an operating point with a high false-alarm rate and that a baseline can match F1 while detecting very few labeled attacks. We share that concern about single aggregate metrics but locate the problem in the split design and in host-specific content artifacts.

Evaluation leakage and shortcut learning.. Geirhos et al. (2020) define shortcut learning as using decision rules that succeed on the training distribution without solving the intended task, and recommend testing under a deliberately shifted condition. Shortcuts can arise early in training (Murali et al., 2023) and follow from gradient-based learning favoring simple predictive features (Hermann et al., 2023). In graph models, degree can act as a shortcut when node features are weak, because neighborhood aggregation encodes it implicitly; small-sample and representation effects in GNN-IDS are discussed by Yehezkel and Elyashiv (2022); Friji et al. (2023); Tran and Park (2024). GNN-based detectors are also open to evasion (Yan et al., 2024).

Explainability and topology dependence.. GNNExplainer (Ying et al., 2019) and PGExplainer (Luo et al., 2020) produce soft edge and feature masks, and Integrated Gradients (Sundararajan et al., 2017; Kokhlikyan et al., 2020) attributes a prediction to input features. These outputs are post-hoc and are not interventions; we therefore treat them as supporting evidence and rely on counterfactual experiments for claims about topology.

Temporal detection.. Temporal link-prediction methods such as Euler (King and Huang, 2023) detect lateral movement from graph snapshots, which motivates our snapshot construction for the strict temporal protocol.

3. Methodology

3.1. Datasets and graph construction

Table 1 summarizes the datasets. **SCADANet** is a labeled packet capture of 534,841 packets with 13 attack classes plus normal traffic. The raw capture has 60 columns and 84.3% attack traffic (451,099 of 534,841 flows). The column audit in the Week 6–8 notebooks removed 9 dead or constant, 2 label-leaking and 24 identifier or free-text columns, held out 2 high-cardinality columns and used 23 (10 numeric, 13 categorical). These figures come from the notebook outputs and not from the final result files. The graph has one node per IP address (6,414 nodes) and one edge per observed directed IP pair (6,535 unique pairs at the end of the capture). `tcp_syn_flood` has 6,286 distinct source IPs and `udp_flood` six; every other attack class has one or two. Node input features are a constant vector, so that node identity and raw degree are not supplied. Each flow is classified from its two endpoint embeddings and an edge-attribute vector of 218 dimensions on Track B and 204 on the strict temporal protocol (categorical vocabularies and scaling are fitted on the training rows of each protocol, so the dimension differs).

BATADAL (`BATADAL_dataset04.csv`, 4,177 hourly rows; 43 sensor and actuator variables, per the Week 5–8 notebooks) is converted to sliding windows of 24 h with a 6 h stride, 693 windows in total. The graph topology is fixed: nodes are the sensor variables and edges connect variables with a Pearson correlation of at least 0.7 ($p < 0.05$), estimated on the attack-free rows of the training portion. Node features vary per window (mean, standard deviation and z-score against the sensor's attack-free baseline). A window is anomalous if it contains attack rows, and a graph-level classifier predicts the window label.

An earlier exploratory component built a security knowledge graph from ICS-Flow (Dehlaghi-Ghadim et al., 2023) (44,156 flows), following Kiesling et al. 2019 and the MITRE ATT&CK ICS matrix (MITRE, 2024). The constructed knowledge graph has 7,685 nodes and 15,402 edges; the edge export loaded into Neo4j in the Week 5–12 notebooks contained 7,733 edges, so we do not claim 15,402 edges in Neo4j. The component supports attack-path queries but was not part of the controlled experiments reported here, and these figures come from notebook outputs and not from the final result files; we draw no detection conclusions from it.

3.2. Split protocols and audit

SCADANet is evaluated under four protocols. **Track A** holds out 20% of IP addresses and evaluates flows touching a held-out address. **Track B** holds out flows at random while the same IPs appear on both sides; the split depends on the seed, and training weights are inverse-frequency by attack subtype. **Strict causal temporal** sorts flows chronologically (stable sort with a content tiebreak), divides them into 20 equal-count windows (equal-width time bins put almost all flows into a handful of windows, as noted in the Week 10 notebook; the exact concentration was not recorded in the final results), trains on windows 0–13 with only their cumulative topology, and evaluates each test window using only edges observed in the windows before it. **Fixed-training topology** is a variant that uses only the topology known at the end of the training period. The earlier protocol, in which the full test-period topology was visible, is kept as a historical reference.

BATADAL uses 5-fold windowed cross-validation as the static reference. Because consecutive windows overlap by 18 h, the temporal protocol uses a **purged chronological split**: windows whose raw rows fall inside both periods are removed (4 of 693 windows), and the audit records zero raw-row overlap. We report 60/40, 70/30 (primary) and 80/20 splits.

For every protocol a split audit records attack-class coverage, the fraction of test hosts and IP pairs also present in training, the fraction of test nodes and edges unseen in training, and the raw-row overlap (Table 3).

3.3. Models and baselines

All models receive the same per-flow content features and the same split. Non-graph baselines are logistic regression, an MLP and XGBoost. The XGBoost baseline is labeled `tree_xgboost` in the result files. In the code the backend is an `XGBClassifier` when the `xgboost` package is importable and a scikit-learn `HistGradientBoostingClassifier` otherwise, and the result name is built from the backend selected at run time (`tree_` followed by the backend name), so the label records an XGBoost run. The exact `xgboost` package version was not recorded. Its learning rate is fixed at 0.1 and the search covers maximum depth $\{4, 6\}$ and number of estimators $\{100, 200\}$. Graph models are GraphSAGE (reference), GCN and GAT edge classifiers with two message-passing layers on the SCADANet graph; for BATADAL, window-level GraphSAGE and GCN classifiers are compared with an MLP on flattened window features. On Track B the MLP has 16,232 parameters on average and the GNNs 19,202 to 36,133 (Table 10).

Table 1: Dataset and graph summary for the final experiments.

Dataset	Unit	Samples	Nodes	Edges	Attack classes	Main protocols
SCADANet	flow (packet)	534,841	6,414 IPs	6,535 IP pairs	13 + normal	Track A/B, strict temporal
BATADAL	24 h window, 6 h stride	693 windows	43 variables	fixed correlation graph, $\lvert r\rvert \geq 0.7$	binary	static CV, purged temporal

3.4. Counterfactuals

Six conditions share splits and seeds. **A** content only (no graph); **B** topology only (true graph, constant dummy edge attribute); **C** full model; **D** random rewiring (same node and edge counts); **E** degree-preserving rewiring by double-edge swaps (64,160 swaps requested on the seed-42 Track B graph); **F** no message passing. Rewiring statistics are saved to confirm the graph was altered as intended. Feature counterfactuals permute `Protocol_TCP`, `Tcp_flags_reset_Set`, `frame_len`, the three jointly, and all content features.

3.5. Host normalization

Earlier analysis traced about 95% of Track B false positives to two hosts, 192.168.119.140 and .141. The host-normalization runs z-score three numeric columns (`frame_len`, `ip_len`, `Tcp_window_size`) against the mean and standard deviation of the source host in the training rows, with a global training-period fallback for hosts with fewer than 20 training samples, and append one per-host flag-rate feature (`host_rate_Tcp_flags_reset_Set`). The runs use a 16-dimensional hidden layer and learning rate 0.003, with 400 epochs on Track B and 300 epochs on the temporal split, and hold out 15% of the training portion as a validation set for threshold selection. Four conditions are compared on identical splits: H1 original model at threshold 0.5; H2 original model with a validation-selected threshold; H3 host-normalized model at 0.5; H4 host-normalized model with a validation-selected threshold.

3.6. Metrics and statistics

We report precision, recall, F1, AUPRC, false-positive rate on normal traffic (FPR), per-attack recall, expected calibration error (ECE) and Brier score. Aggregates are means over ten seeds with 95% t-intervals. Paired comparisons use a paired t-test with Cohen's d on seed-matched differences, with the mean difference and its interval reported alongside the p-value.

4. Experimental Setup

Seeds and runs. Ten seeds (42, 7, 123, 1, 2024, 13, 21, 99, 2025, 314). On Track B the seed also changes the split; the temporal split is one fixed partition, so temporal intervals reflect training randomness only. **Training.** 300 epochs per configuration for the SCADANet baselines and counterfactuals, 100 for BATADAL, and for the host-normalization runs 400 on Track B and 300 on the temporal split. Track B uses per-sample subtype-balanced weights and the temporal protocol binary class weights. **Search space (baselines and counterfactuals).** Hidden dimension $\{32, 64\}$, learning rate $\{10^{-3}, 5 \times 10^{-3}\}$, dropout $\{0, 0.2\}$; XGBoost as in Section 3.3; logistic regression $C \in \{0.1, 1\}$. Selection uses validation data only. **Thresholds.** Chosen on a validation subset of the training period over the grid $0.05, 0.10, \ldots, 0.95$ and applied once to test predictions. Some tables use a fixed 0.5 instead; each such table or row is labeled. **Splits.** Track B (seed 42): 359,418 training, 63,426 validation and 105,711 test flows. Strict temporal: 318,231 training, 56,158 validation and 160,452 test flows. BATADAL purged 70/30: 484 training-period windows, 4 purged windows and 205 test windows. In B3 the last 72 training-period windows (15%) form the validation block and 405 windows are used for fitting. The remaining 7 windows (405+72+7=484) are purged between fit and validation so that no fit window shares raw rows with the validation block: a fit window must end at least one full window before the validation block starts. **Software.** The result files record Python 3.12.3, PyTorch 2.14.1+cu130, PyTorch Geometric 2.8.0.post1, NumPy 2.5.3, pandas 3.0.6, scikit-learn 1.9.1 and NetworkX 3.6.1. The exact `xgboost` package version is not recorded. **Hardware.** `run_config.json` sets 4 CPU threads; the processor and GPU model, and whether a GPU was used, were not recorded, so the timings in Table 10 are indicative only. **Provenance.** `run_config.json` records the Git commit hash (`ef0efd2c...`), a fingerprint of the source files and SHA-256 hashes of both input CSV files. Among the 434 per-seed result JSON files, 414

carry the commit and 254 carry the source fingerprint, so not every result file carries every field. The baseline (B1–B3) and counterfactual (A–F) result files carry the commit, source fingerprint, split-manifest identifiers and library versions. `validation_report.json` records 312 checks (21 tables, 26 figures, ten seeds), all passed; they include per-file provenance-field checks, per-model and per-seed completeness checks and split-audit checks, and they do not test that the recorded commit exists in the repository history.

GraphSAGE strict-temporal configurations.. Several strict-temporal GraphSAGE F1 values appear below because they come from different configurations, and they must not be compared as one setup (Table 2). Comparisons are made only within a configuration.

5. Results

5.1. RQ1: effect of the evaluation protocol

Table 3 shows that the three SCADANet protocols answer different questions. Track A can only be evaluated on `tcp_syn_flood` and `udp_flood` (1,263 and 14 test flows): the other classes have one or two source IPs, so each is wholly in training or wholly in test, and recall for them is undefined rather than measured. The Track A F1 of 0.993 recorded in the Week 8 notebook (enriched model; 0.992 for the baseline) describes only these two classes; Track A was not rerun in the final result set (only audited), and the figure is not used for any comparison below. Track B covers 12 of 13 classes (`tcp_syn_flood` is excluded) but shares every host and 99.6% of IP pairs between training and test, so it measures recognition of known attacker behavior and not generalization to new hosts. The strict temporal split has almost no host overlap (0.25%) and contains 11 classes, because `os_scan` and `udp_flood` occur only in the training period.

Moving from Track B to strict temporal evaluation lowers F1 sharply (Table 4). With identical tuning, GraphSAGE falls from 0.985 to 0.683 ($\Delta = -0.302$), MLP from 0.984 to 0.686 (-0.298) and XGBoost from 0.986 to 0.687 (-0.299). GCN is the exception (0.986 to 0.914, -0.072). A comparison within the fixed-threshold reference configuration isolates the effect of the future-topology leak: the earlier protocol, with future topology visible, gives F1 0.627 [0.596, 0.659], the strict causal protocol 0.630 [0.599, 0.660] and the fixed-training-topology variant 0.630 [0.599, 0.660], with recall 0.468 in each. Removing the leak did not change the result, so the gap is not an artifact of that leak. Nor is it explained by unseen classes: every attack class in the strict temporal test set also occurs in the training period. Whether the remaining gap is due to distribution shift, to threshold choice under shift, or to the training configuration cannot be separated with the present experiments (Section 7).

On BATADAL the purge changes the result modestly: the earlier overlapping split gives a ten-seed mean F1 of 0.765 and the purged 70/30 split 0.738 [0.716, 0.760], both at a fixed 0.5 threshold. The 0.778 reported for the earlier split in the earlier manuscript is its seed-42 value. F1 across the three purged splits ranges from 0.738 to 0.803. The static reference of 0.857 (0.8567) is the seed-42 run in the final result set (`references/batadal_static_cv_seed42.json`); an earlier run with the same seed in a different software environment, with no commit recorded, gave 0.862 (0.8621, the value in the earlier manuscript) and differs in one of the five folds. BATADAL fails differently from SCADANet: recall stays near 0.97–0.99 and precision falls (0.59–0.69), so the cost is false alarms and not missed attacks. Only a small number of anomalous windows is available in each test split, which limits how finely the splits can be compared.

5.2. RQ2: does topology contribute?

Baselines.. On Track B all six models lie within 0.007 F1 of each other (Table 5). XGBoost has the highest F1 (0.9860) and the lowest FPR (0.123); GraphSAGE has the highest FPR (0.148). Paired across seeds, GraphSAGE exceeds the MLP by 0.0007 F1 (95% CI 0.0006–0.0007, $d = 6.3$) and trails XGBoost by 0.0012 (CI -0.0015 to -0.0009, $d = -2.6$). These differences are statistically detectable because seed variance is small, but their size has no operational meaning. On this protocol GraphSAGE provides no practically meaningful improvement over a content-only classifier. It is also far more expensive: 28,437 parameters on average and 1,776 s mean training time, against 6.6 s for XGBoost (Table 10).

Under strict temporal evaluation the models split. MLP, XGBoost and GraphSAGE reach F1 0.68–0.69, whereas GCN reaches 0.914. The GCN result is reproducible across all ten seeds (F1 0.890–0.922, threshold 0.05 on every seed, `vuln_scan` recall 0.876–0.877 on every seed) and is unexplained (Section 5.2). MLP, GCN and XGBoost selected the minimum grid threshold (0.05) on every seed and GraphSAGE on nine of ten (0.10 on seed 1), using a validation set drawn from the training period (Section 7). On BATADAL, GraphSAGE (0.687) outperforms GCN (0.639) at equal recall. These B3 values differ from the purged 70/30 row of

Table 2: GraphSAGE on the SCADANet strict causal temporal split: the configurations behind each reported F1 (10 seeds, mean).

Label	Training and threshold	Source	F1
Fixed-threshold reference (T1)	300 epochs, lr 0.003, binary class weights, no hyperparameter search, threshold fixed at 0.5	Table 4 (rows marked †)	0.6295
Condition C (A–F study)	grid search, validation-selected threshold (0.10 for seed 42)	Table 6	0.6759
Tuned baseline (B2)	grid search, validation-selected threshold (0.05 on nine seeds, 0.10 on seed 1)	Tables 4 and 5	0.6829
Host runs H1 / H2	16 hidden units, lr 0.003, 300 epochs, 15% of the training period held out; threshold 0.5 (H1) or validation-selected (H2)	Table 9	0.6432 / 0.6873

Table 3: Split audit. Overlap is the share of test hosts or IP pairs also present in training. SCADANet has 13 attack classes.

Dataset	Protocol	Test size	Attack classes in test	Host / pair overlap	Main warning
SCADANet	Track A (IP-held-out)	10,940	2 of 13	0.3% / 0.0%	only `tcp_syn_flood`, `udp_flood`
SCADANet	Track B (flow-level)	105,711	12 of 13	100% / 99.6%	same hosts and pairs on both sides
SCADANet	Strict temporal	160,452	11 of 13	0.25% / 0.5%	99.75% of test nodes unseen
BATADAL	Static 5-fold CV	138–139 per fold	binary	—	temporal overlap recorded as 1.0
BATADAL	Earlier temporal 70/30	208	binary	—	overlap 0.014 (3 windows)
BATADAL	Purged 70/30	205	binary	—	overlap 0

Table 4 (0.738) because B3 fits on 405 windows with a validation-selected threshold, whereas that row uses all 484 training-period windows and a fixed 0.5 threshold. The flattened-window MLP predicts no anomalies (F1 0) and its AUROC is 0.02 on every seed, which indicates an inverted ranking on the purged test set. We could not determine whether this reflects a covariate shift between training and test windows or a defect in the flattened representation, so this result is reported but not interpreted, and the BATADAL comparison with a flattened-window MLP should not be relied on.

Counterfactuals on Track B.. Removing or destroying the graph does not hurt (Table 6). Replacing the true topology with a random graph or a degree-preserving rewiring leaves F1 at 0.9854, slightly above the true-graph model (+0.0006; the interval excludes zero but the size is negligible), and disabling message passing gives 0.9860. Condition A is the same model as the MLP baseline; it and the no-message-passing model reach the same F1 as the full model. A topology-only model, which sees no flow content, still reaches F1 0.980 and AUPRC 0.973. Feature interventions point the same way (Table 7): permuting `Protocol_TCP` lowers F1 by 0.0084 and raises FPR by 0.099, permuting `frame_len` lowers F1 by 0.0166 and lowers FPR by 0.064, permuting `Tcp_flags_reset_Set` changes nothing, and shuffling all content features lowers F1 by 0.063 with FPR rising to 0.357. On Track B, performance is carried primarily by content, and we find no practically meaningful advantage from the true topology.

This finding is specific to this protocol and graph. The SCADANet graph has 6,414 nodes and 6,535 pairs with a maximum degree of 6,293 and a mean degree of 2.04, so it is almost a star around the dominant hosts. In the seed-42 rewiring metadata, random rewiring reduces the maximum degree to 9 and fragments the graph into 1,014 components, whereas degree-preserving rewiring, which operates on the deduplicated undirected graph (6,416 edges, maximum degree 6,289), keeps the hub (8 components). Neither changes the full model's F1, but the controls have limited power on a graph of this shape. A topology-only model can still separate classes on Track B (F1 0.980), and we have not established whether that signal reflects relational structure or the position of hosts in this particular layout.

Counterfactuals under strict temporal evaluation.. Here the picture differs. The topology-only model reaches F1 0.915 against 0.676 for the full model (Δ = +0.239, CI 0.212–0.266), with recall 0.944 on every seed. On nine of ten seeds F1 is 0.923 with identical true positives (78,663); seed 123 has the same recall but 25,641 false positives (FPR 0.333, F1 0.839). Its per-attack recall (Table 8) shows an attack-dependent pattern: it detects `vuln_scan` (0.997) which every condition with content features nearly

Table 4: Track B versus strict causal temporal evaluation for SCADANet and static versus purged temporal evaluation for BATADAL. For SCADANet, ΔF1 is the strict temporal value minus Track B for the same model and tuning. For BATADAL, ΔF1 is the purged temporal value minus the seed-42 static 5-fold CV reference. Rows marked † use the fixed 0.5 threshold of the reference configuration (Table 2). Values are ten-seed means with 95% CI for F1, except the static BATADAL reference (seed 42); the BATADAL purged rows also use a fixed 0.5 threshold.

Dataset	Model / protocol	Static F1	Strict temporal F1 [95% CI]	ΔF1	Recall	FPR
SCADANet	MLP	0.9841	0.6858 [0.6834, 0.6882]	−0.2983	0.546	0.050
SCADANet	XGBoost	0.9860	0.6869 [0.6826, 0.6912]	−0.2991	0.580	0.117
SCADANet	GraphSAGE (B2)	0.9848	0.6829 [0.6728, 0.6930]	−0.3019	0.547	0.061
SCADANet	GCN (B2)	0.9859	0.9140 [0.9055, 0.9224]	−0.0719	0.937	0.123
SCADANet	GraphSAGE, earlier protocol†	—	0.6274 [0.5962, 0.6586]	—	0.468	0.023
SCADANet	GraphSAGE, strict causal†	—	0.6295 [0.5989, 0.6601]	—	0.468	0.018
BATADAL	GraphSAGE, purged 60/40	0.8567	0.7805 [0.7503, 0.8107]	−0.076	0.984	0.069
BATADAL	GraphSAGE, purged 70/30	0.8567	0.7378 [0.7156, 0.7600]	−0.119	0.986	0.079
BATADAL	GraphSAGE, purged 80/20	0.8567	0.8026 [0.7676, 0.8375]	−0.054	0.971	0.082

Table 5: Model baselines (10 seeds, mean; F1 with 95% CI). Thresholds are validation-selected. BATADAL: purged 70/30.

Protocol	Model	Recall	F1 [95% CI]	AUPRC	FPR
Track B	Logistic regression	0.9823	0.9793 [0.9773, 0.9812]	0.9972	0.127
Track B	XGBoost	0.9950	0.9860 [0.9858, 0.9862]	0.9988	0.123
Track B	MLP	0.9942	0.9841 [0.9837, 0.9846]	0.9979	0.139
Track B	GCN	0.9971	0.9859 [0.9857, 0.9861]	0.9985	0.136
Track B	GraphSAGE	0.9970	0.9848 [0.9844, 0.9852]	0.9983	0.148
Track B	GAT	0.9941	0.9841 [0.9836, 0.9846]	0.9979	0.140
Strict temporal	MLP	0.546	0.6858 [0.6834, 0.6882]	0.831	0.050
Strict temporal	XGBoost	0.580	0.6869 [0.6826, 0.6912]	0.889	0.117
Strict temporal	GraphSAGE (B2)	0.547	0.6829 [0.6728, 0.6930]	0.916	0.061
Strict temporal	GCN (B2)	0.937	0.9140 [0.9055, 0.9224]	0.965	0.123
BATADAL purged	MLP (flattened window)	0.000	0.000 [0.000, 0.000]	0.056	0.054
BATADAL purged	GraphSAGE	1.000	0.6866 [0.6757, 0.6974]	0.655	0.104
BATADAL purged	GCN	1.000	0.6389 [0.6244, 0.6533]	0.752	0.129

misses (0.014–0.016), but recalls `insider_threat`, `modbus_abuse` and `modbus_fdi` at 0.000. The GCN baseline also recovers `vuln_scan` (0.877) while keeping the other classes at the level of GraphSAGE, including `modbus_abuse` (recall 0 on nine seeds and 1.0 on seed 21, giving a ten-seed mean of 0.10), which only XGBoost retains on every seed. When content features are present, the full GraphSAGE model did not exploit whatever the topology contributes to `vuln_scan`. We therefore do not conclude that topology is uninformative: its utility is context-dependent and attack-dependent, and the full GraphSAGE model did not consistently exploit it.

Unresolved findings. The GCN and topology-only results are confirmed, reproducible results whose mechanism is unresolved. Topology-only controls with random rewiring and with degree-preserving rewiring were not run, so genuine relational learning cannot be distinguished from degree, hub-layout, novelty (99.75% of test nodes are unseen, and a scanning host creates new IP pairs in every window) or other topology-associated shortcuts; these are candidates, not findings. Why the GCN differs from GraphSAGE under identical tuning, with no configuration difference apparent in the result files, is likewise unexplained.

5.3. RQ3: robustness and operational implications

Per-attack behavior.. Table 8 shows that per-attack behavior depends on the configuration. In the tuned B2 setting, `vuln_scan` makes up 37,777 of the 83,340 attack flows in the test period (45.3%), and its share of all flows rises from 0.83% in training to 23.5% in test. A model that misses it entirely has a recall ceiling of $1 - 0.453 = 0.547$, which is what the tuned MLP (0.546) and GraphSAGE (0.547) reach, so for these models the missed recall is largely a `vuln_scan` failure. This does not hold for every configuration: the fixed-threshold reference has recall 0.468, below that ceiling, and it also misses `apt_exfil` (recall 0.231), `insider_threat` (0.284), `sql_injection` (0.496), `modbus_fdi` (0.565) and `modbus_abuse` (0.000). The tuned models recover most of these classes, which suggests that threshold choice and training configuration

Table 6: Topology and content counterfactuals (10 seeds, mean). Δ is relative to the full GraphSAGE model (condition C); the bracket is the 95% CI of ΔF1. Strict temporal rows use validation-selected thresholds (the A–F configuration in Table 2).

	Content	True topology	F1	AUPRC	FPR	ΔF1 [95% CI]
Track B						
A Content only	yes	no	0.9841	0.9979	0.139	−0.0007 [−0.0008, −0.0006]
B Topology only	no	yes	0.9802	0.9731	0.137	−0.0046 [−0.0049, −0.0043]
C Full GraphSAGE	yes	yes	0.9848	0.9984	0.147	0
D Random topology	yes	no	0.9854	0.9983	0.132	+0.0006 [0.0003, 0.0009]
E Degree-preserving	yes	no	0.9854	0.9986	0.126	+0.0006 [0.0003, 0.0009]
F No message passing	yes	partial	0.9860	0.9984	0.135	+0.0012 [0.0009, 0.0015]
Strict temporal						
A Content only	yes	no	0.6858	0.8314	0.050	+0.0099 [−0.0054, 0.0252]
B Topology only	no	yes	0.9147	0.8799	0.132	+0.2388 [0.2117, 0.2658]
C Full GraphSAGE	yes	yes	0.6759	0.9095	0.079	0
D Random topology	yes	no	0.6827	0.8504	0.060	+0.0068 [−0.0084, 0.0220]
E Degree-preserving	yes	no	0.6926	0.9266	0.040	+0.0167 [0.0019, 0.0314]
F No message passing	yes	partial	0.6762	0.8645	0.092	+0.0003 [−0.0148, 0.0153]

Table 7: Track B feature interventions on GraphSAGE (10 seeds). The unmodified baseline in this run has FPR 0.1495; ΔFPR is relative to it.

Intervention	F1	ΔF1	ΔFPR
None	0.9848	0	0
`Protocol_TCP`	0.9764	−0.0084	+0.099
`Tcp_flags_reset_Set`	0.9848	0.0000	0.000
`frame_len`	0.9683	−0.0166	−0.064
Top three jointly	0.9777	−0.0072	+0.051
All content shuffled	0.9214	−0.0634	+0.207

contribute to the temporal result alongside any distribution shift; the present data do not allow us to apportion the effect between these causes. Training sample size does not predict which classes are learned: in the tuned setting `apt_exfil`, with 4 training flows, reaches full recall and `vuln_scan`, with 3,093, fails, whereas in the fixed-threshold reference `apt_exfil` reaches 0.231. For `modbus_abuse` the models split: XGBoost has recall 1.0 on every seed, whereas for the tuned neural models recall was 0 on nine seeds and 1.0 on seed 21, giving a ten-seed mean of 0.10 (the fixed-threshold reference gives 0.000). The same seed flips GraphSAGE, GCN, the MLP and conditions A and C together. The thresholds do not differ (0.05 throughout, except GraphSAGE seed 1 at 0.10), and per-flow probabilities and the fit and validation composition of each seed's carve were not saved, so the mechanism is unresolved. We have not isolated the cause of the other class-level differences.

One caution applies to the Track B reference used for per-attack deltas in the fixed-threshold analysis. The matched static control (same architecture, epochs, learning rate and fixed 0.5 threshold) already has `vuln_scan` recall of 0.011 and `modbus_abuse` recall of 0.000, while the tuned Track B GraphSAGE recalls `vuln_scan` at 0.992. The size of the per-class drop for `vuln_scan` therefore depends on the reference configuration and threshold and should not be read as a pure effect of time. The aggregate drop in Table 4 is computed between runs with matched tuning and does not have this problem.

Host-specific false positives and mitigation.. H1: original model, threshold 0.5; H2: original, validation-selected threshold; H3: host-normalized, 0.5; H4: host-normalized, validation-selected threshold. Table 9 separates the two mitigations. On Track B, threshold calibration accounts for nearly all of the improvement: FPR falls from 0.277 to 0.144 (H2) with recall changing from 0.9997 to 0.9957, and false positives on host .141 drop from 2,068 to 6. Host normalization alone (H3) lowers FPR only from 0.277 to 0.264 and leaves both hosts' false positives essentially unchanged (2,317 and 2,055); adding it to calibration (H4) lowers FPR by a further 0.009 (to 0.135). The false positives on host .140 persist under every condition (2,316 to 2,230). A paired comparison of H1 and H4 shows an FPR reduction of 0.142 (CI 0.138–0.146), but it combines both mitigations, and the threshold accounts for 0.133 of it (H1 minus H2). The hypothesis that host-relative features cor-

Table 8: Strict temporal recall by attack class. SAGE-T1 is the fixed-threshold reference (threshold 0.5); the other columns are the tuned B2 models (validation-selected thresholds). Counts are for the strict causal split. For `modbus_abuse`, SAGE (B2) and GCN (B2) recall was 0 on nine seeds and 1.0 on seed 21, giving a ten-seed mean of 0.10; XGBoost (B2) recall was 1.0 on every seed.

Attack	Train	Test	SAGE-T1	SAGE (B2)	XGBoost (B2)	GCN (B2)
`vuln_scan`	3,093	37,777	0.006	0.016	0.073	0.877
`modbus_abuse`	98	498	0.000	0.100	1.000	0.100
`modbus_fdi`	298	2,636	0.565	0.976	1.000	0.975
`insider_threat`	56	1,446	0.284	0.967	1.000	0.966
`apt_exfil`	4	2,421	0.231	1.000	1.000	1.000
`sql_injection`	23	360	0.496	1.000	1.000	1.000
`port_scan`	174	1,387	0.888	1.000	1.000	1.000
`tcp_syn_flood`	784	5,502	0.661	1.000	0.999	1.000
http, icmp, tcp_ack floods	22,529	31,313	0.992–1.000	1.000	1.000	1.000

Table 9: Host-normalization and threshold mitigation, conditions H1–H4 (10 seeds, mean). Total FP is the number of false positives on normal test traffic; top-2 share is the fraction caused by hosts .140 and .141 (derived from the per-host counts).

Protocol	Cond.	F1 [95% CI]	Normal FPR	Total FP	Top-2 share	FP .140 / .141
Track B	H1	0.9745 [0.9742, 0.9747]	0.277	4,639	94.5%	2,316 / 2,068
Track B	H2	0.9845 [0.9840, 0.9849]	0.144	2,407	91.0%	2,185 / 6
Track B	H3	0.9755 [0.9753, 0.9758]	0.264	4,427	98.8%	2,317 / 2,055
Track B	H4	0.9854 [0.9852, 0.9857]	0.135	2,265	98.8%	2,230 / 8
Strict temporal	H1	0.6432 [0.6211, 0.6653]	0.023	1,736	—	—
Strict temporal	H2	0.6873 [0.6809, 0.6936]	0.046	3,532	—	—
Strict temporal	H3	0.6470 [0.6322, 0.6617]	0.019	1,447	—	—
Strict temporal	H4	0.6781 [0.6597, 0.6965]	0.072	5,527	—	—

rect a host-specific content artifact is therefore not supported by these data. Calibration is a usable mitigation for host .141 and not for host .140. Because H2 uses the same model as H1, the mean counts imply that at least 99.7% of host .141's false positives score between 0.5 and the calibrated threshold, whereas at least 94% of host .140's (2,185 of 2,316 on average) score above it. Why host .140 behaves differently was not established, because per-flow scores and features were not saved.

The mitigations also change what is detected. On Track B, `os_scan` recall falls substantially: 0.9967 (H1), 0.4974 (H2), 0.9959 (H3) and 0.4958 (H4). The drop appears in both conditions that use the validation-selected threshold (mean 0.835 for H2 and 0.825 for H4) and in neither condition that uses 0.5, so it is associated with validation-selected threshold calibration and not with host normalization. About half of the `os_scan` flows are lost on every seed (SD 0.025 across seeds); we have not examined why, since per-flow scores were not saved. `vuln_scan` recall falls from 0.9972 to 0.9776 (H2) and 0.9821 (H4), `port_scan` by 0.001 at most, and the other classes change by 0.001 or less. Part of the lower FPR was thus obtained at a substantial cost in one attack class. Under strict temporal evaluation neither mitigation improves matters consistently. Calibration (H2) raises F1 to 0.687 and raises recall on `insider_threat` (0.382 to 0.962) and `modbus_fdi` (0.756 to 0.971), but doubles total false positives (1,736 to 3,532); H4 raises them to 5,527. The FPR change from H1 to H4 has an interval that includes zero (−0.049, 95% CI −0.104 to 0.006, $p = 0.073$). `modbus_abuse` recall is 0.000 and `vuln_scan` at most 0.015 under all four temporal conditions.

Seed stability and calibration.. Seed-to-seed variation is small relative to the effects in Tables 5–9 on Track B (F1 standard deviation 0.0003–0.0006 for every model except logistic regression, 0.0028) and larger under strict temporal evaluation (0.003–0.043 across the temporal tables, varying with the configuration). Calibration is good on Track B (ECE 0.028–0.034 and Brier 0.031–0.035 for XGBoost, GraphSAGE and MLP) and poor under temporal shift in the fixed-threshold reference configuration (GraphSAGE ECE 0.270, Brier 0.259), so temporal scores should not be read as probabilities. False alarms per hour, event-level recall and time to detect are reported as not available in the result files: the Track B sample has no continuous exposure denominator and no verified event identifiers are available, and reconstructing events heuristically would not be ground truth.

Explainability.. Integrated Gradients on 1,500 flows per prediction group (150 per seed, ten seeds) ranks `Protocol_Modbus/TCP`, `Protocol_TCP` and `Tcp_flags_reset_Not set` highest for true and false positives, `Protocol_TCP` and `Tcp_flags_reset_Set` for false negatives, and `ip_ttl` for true negatives. Cross-seed rank correlation is 0.92–0.97, and the top-three overlap (Jaccard) is 0.52 for false negatives, 0.55 for false positives, 0.69 for true positives and 0.90 for true negatives. Rankings are therefore stable in order but not in the identity of the top three. The intervention test is the stronger evidence: permuting `Protocol_TCP`, which is in the top three for every prediction group, degrades the model, as does permuting `frame_len`, whereas `Tcp_flags_reset_Set`, which an earlier 40-flow analysis had listed as a leading driver of false positives, has no measurable functional effect (Table 7). Attribution rank alone would have overstated that feature.

6. Discussion

What inflated the metrics.. Three sources are visible. The IP-held-out split covers 2 of 13 classes, so its earlier headline F1 of 0.993 says nothing about the other eleven. The flow-level split lets the same hosts and 99.6% of IP pairs appear on both sides and is close to saturated: even logistic regression reaches F1 0.979. Third, the threshold: on Track B, moving from 0.5 to a validation-selected value changes FPR from 0.277 to 0.144 with no change to the model. Each correction removes a layer of apparent performance, and the largest remaining effect is temporal.

When topology helps and when content dominates.. On Track B, content dominates and the true topology provides no practically meaningful performance advantage, so the GNN's added cost (about 270 times the training time of XGBoost) buys no practically meaningful gain. Under strict temporal evaluation the evidence is mixed. A topology-only model recovers an attack class that content-bearing models lose and also loses three classes that content-bearing models detect, and a GCN recovers the same class. This does not show that topology is useless, and it does not show that topology is reliably useful: the information in this graph appears complementary to content for one attack class, and the full GraphSAGE model did not combine the two. Whether a model that keeps both exists is untested.

Attack-specific failure.. A single F1 conceals the structure of the failure. In the tuned setting the aggregate temporal recall of 0.55 is arithmetically the share of attack flows outside `vuln_scan`, while the fixed-threshold reference misses several further classes, and XGBoost and the neural models differ on `modbus_abuse` (1.00 on every seed against 0 on nine seeds and 1.0 on seed 21, a ten-seed mean of 0.10) despite similar F1. Per-attack recall should accompany any temporal benchmark, together with the threshold and training configuration.

Mitigation.. Threshold calibration is cheap and effective for one of the two dominant hosts, and host normalization adds little beyond it. The remaining false positives on host .140 and the drop in `os_scan` recall under the validation-selected threshold show that a lower FPR is not free. Mitigations should be reported with per-attack recall.

Implications for GNN-IDS evaluation.. The practices that follow from these data are: report attack-class coverage and identity overlap for every split; compare against content-only and tree baselines under the same split; include random and degree-preserving rewiring controls; report per-attack recall under a causal temporal split with the threshold and training configuration stated; and treat explainer output as a hypothesis to be tested by intervention.

7. Threats to Validity and Limitations

- **Two datasets, both simulated or testbed-derived.** SCADANet's graph is a hub with many leaves and most attack classes have a single source IP. Conclusions about topology depend on that layout, and the rewiring controls have limited power on a near-star graph.
- **Temporal seeds.** The temporal split is one fixed partition, so intervals reflect training randomness and not variation in the split. Track B seeds vary the split.
- **Thresholds and configurations under drift.** Thresholds are selected on a validation subset of the training period, which cannot reflect drift. All SCADANet B2 models selected the grid minimum (0.05) on at least nine of ten seeds, and some tables use a fixed 0.5. GraphSAGE strict-temporal F1 therefore differs across tables (0.6295, 0.6759, 0.6829, 0.6432; Table 2) and comparisons are valid within a configuration only. The effects of distribution shift, threshold choice and training configuration are not separated here.

Table 10: Model size and cost, as recorded in the result files (mean over seeds; processor not recorded).

Model (Track B)	Params	Train (s)	Infer (ms)
Logistic regression	218	15	12.0
MLP	16,232	540	11.4
XGBoost	—	6.6	10.7
GraphSAGE	28,437	1,776	63.8
GCN	19,202	1,789	67.1
GAT	36,133	1,679	68.9

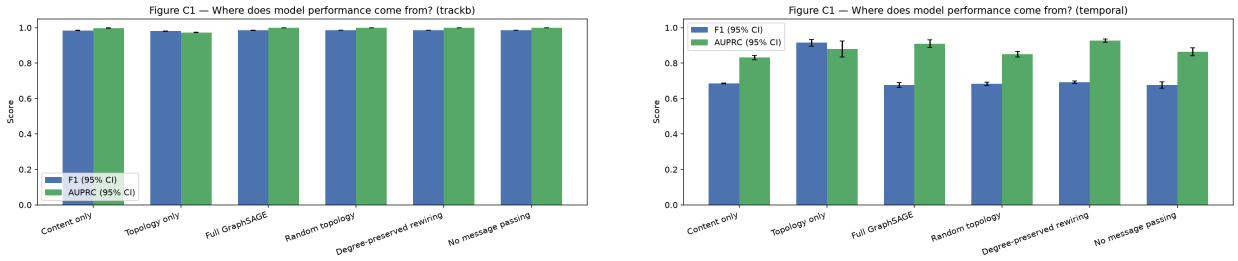


Figure 1: Topology and content counterfactuals on SCADANet, Track B (left) and strict temporal (right), over ten seeds.

- **Unresolved results.** The GCN and topology-only temporal results, the BATADAL flattened-MLP inversion, the `modbus_abuse` difference between XGBoost and the neural models, and the `os_scan` recall reduction under the validation-selected threshold are confirmed or reported but not explained; topology-only random and degree-preserving rewiring controls were not run. The matched Track B control used for per-attack deltas behaves differently from the tuned Track B model.

- **Host .140.** Host .141 is resolved by threshold calibration; host .140 was not resolved by any mitigation, and its cause could not be examined because per-flow scores and features were not saved.

- **BATADAL scale.** Each split has a small number of anomalous test windows and a single attack process. The static reference (0.8567) is a single seed-42 run, whereas the earlier-split value (0.7647) is a ten-seed mean at a fixed threshold, so static and temporal values do not have matched seed aggregation.

- **Operational metrics.** False alarms per hour, event recall and time to detect could not be computed from the available labels.

- **Explainability.** Attributions describe the model, not the attack, and rank stability at the level of the top three features is moderate.

- **Environment and provenance.** The processor and GPU and the exact `xgboost` package version were not recorded; the XGBoost backend is recorded through the `tree_xgboost` result label. The commit and source fingerprint are present in the baseline and counterfactual result files but not in every result file (414 of 434 per-seed files carry a commit and 254 a source fingerprint). The validation report (312 checks, all passed) checks field presence and completeness, not that the recorded commit matches a repository history.

- **Scope.** This paper does not implement attack-path prediction, propagation-likelihood estimation or cyber-to-physical impact modeling, and the knowledge-graph component is exploratory.

8. Conclusion

RQ1. The evaluation protocol changes the result more than the model choice does. Track A covers 2 of 13 attack classes, Track B shares 99.6% of IP pairs between training and test, and strict temporal evaluation lowers F1 by about 0.30 for tuned GraphSAGE, MLP and XGBoost (0.985 to 0.683, 0.984 to 0.686, 0.986 to 0.687). Removing the future-topology leak did not change the result (0.627 versus 0.630 at a fixed threshold). On BATADAL, purging overlapping windows lowers F1 from 0.765 to 0.738 (ten-seed means at a fixed threshold), and F1 under the purged splits is

dataset → graph construction → split protocol → leakage and shortcut audit → model training and baselines → topology/content counterfactuals → strict temporal evaluation → robustness and operational metrics

Figure 2: Evaluation-audit workflow used in this study.

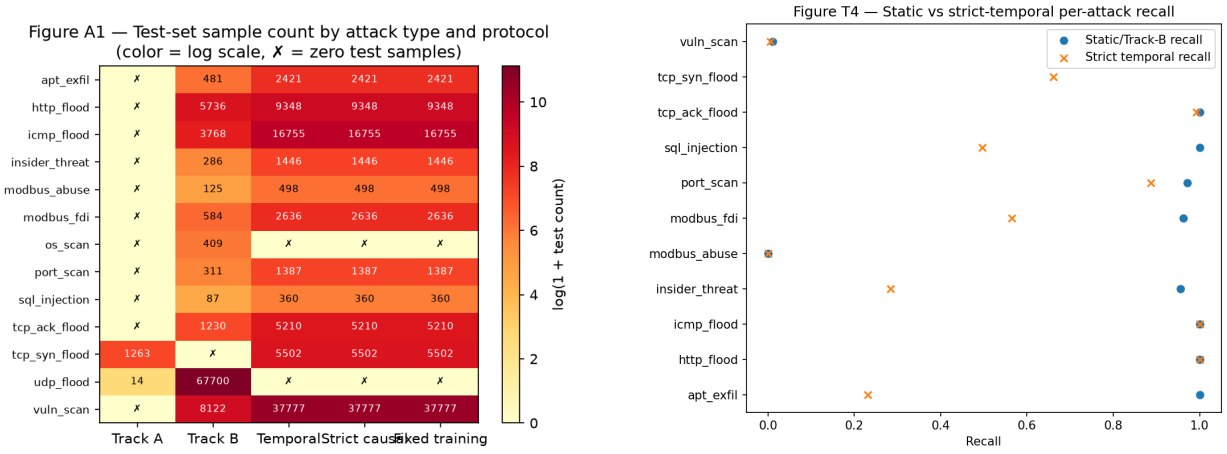


Figure 3: Left: attack-class coverage of the SCADANet split protocols. Right: per-attack recall under static and strict temporal evaluation (fixed-threshold reference; the static markers come from the matched fixed-threshold Track B control).

0.054–0.119 below the static reference (0.857, a single seed-42 run).

RQ2. On the flow-level split the true topology provides no practically meaningful advantage: GraphSAGE (0.985) matches MLP (0.984), XGBoost (0.986) and GCN (0.986), and random or degree-preserving rewiring does not lower F1. Under strict temporal evaluation, topology utility is attack-dependent and context-dependent: topology-only and GCN models detect `vuln_scan`, topology-only misses three other classes, and the full GraphSAGE model did not consistently exploit the available topology. Topology-only rewiring controls were not run, so we cannot yet separate relational information from degree, hub-layout, novelty or other structural effects.

RQ3. The Track B findings are stable across ten seeds (F1 standard deviation ≤0.0006 apart from logistic regression). The temporal gap is stable across initializations, but its attribution to a single attack class holds for the tuned configurations and not for the fixed-threshold reference, and model rankings under temporal shift depend on the threshold and training configuration. Host normalization did not reduce host-concentrated false positives beyond threshold calibration, and the validation-selected threshold substantially reduced `os_scan` recall (0.997 to about 0.50).

Open work: a topology-only control on rewired graphs, threshold selection that accounts for drift, an explanation of the GCN and BATADAL-MLP results, evaluation on further datasets, and the attack-path and physical-impact components that this paper does not claim.

Acknowledgments

This work was carried out as part of an AI Security internship hosted by CNIT / PNTLab Pisa (TeCIP Institute, Scuola Superiore Sant'Anna), under Project 12: "Graph-Based Risk Assessment for Cyber-Physical Systems."

Data and code availability

The result files, configuration files (`run_config.json`), split manifests, split audits and validation report are in the project repository (`AI-Security-Internships-2026/12-graph-based-cyber-physical-risk`); `docs/paper-result-map.md` maps each table and figure to its source file. The input datasets are used as obtained from their original sources, and SHA-256 hashes of both input CSV files are recorded in `run_config.json`. The recorded provenance is limited to what the result files contain (Section 4 and Section 7).

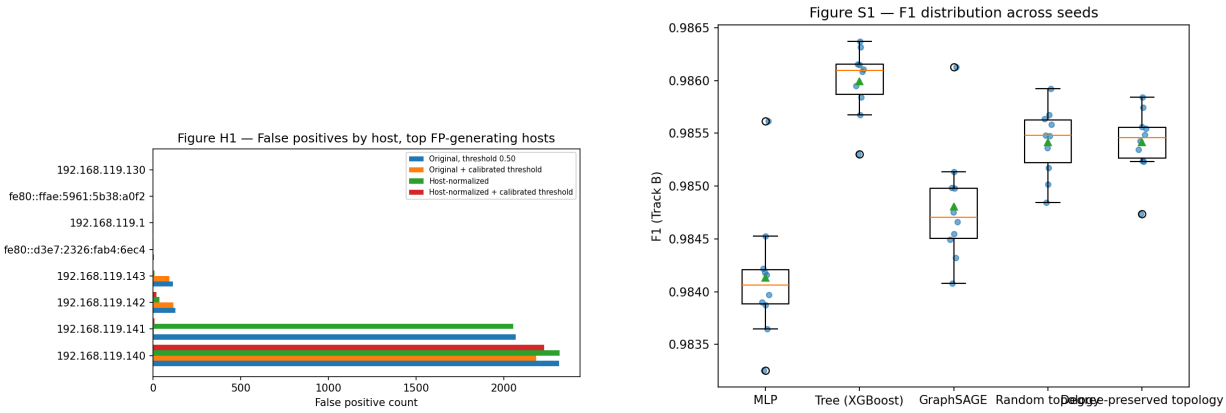


Figure 4: Left: false positives by source host under H1–H4. Right: Track B F1 across ten seeds for the MLP, XGBoost, GraphSAGE and the random and degree-preserved topology conditions.

Appendix A. Reference configuration note

Tables labeled with the fixed-threshold reference (T1) use a model trained for 300 epochs at learning rate 0.003 with binary class weights and a threshold fixed at 0.5. The hidden size of that model is not recorded in the configuration file.

References

Conti, M., Donadel, D., Turrin, F., 2021. A survey on industrial control system testbeds and datasets for security research. IEEE Communications Surveys & Tutorials 23(4), 2248–2294.

Dehlaghi-Ghadim, A., Helali Moghadam, M., Balador, A., Hansson, H., 2023. Anomaly detection dataset for industrial control systems. IEEE Access 11, 107982–107996.

Friji, H., Olivereau, A., Sarkiss, M., 2023. Efficient network representation for GNN-based intrusion detection. In: International Conference on Applied Cryptography and Network Security. Springer, pp. 532–554.

Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., Wichmann, F. A., 2020. Shortcut learning in deep neural networks. Nature Machine Intelligence 2(11), 665–673.

Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. Advances in Neural Information Processing Systems 30.

Hermann, K., Mobahi, H., Fel, T., Mozer, M. C., 2023. On the foundations of shortcut learning. arXiv preprint arXiv:2310.16228.

Kiesling, E., Ekelhart, A., Kurniawan, K., et al., 2019. The SEPSES knowledge graph: An integrated resource for cybersecurity. In: International Semantic Web Conference (ISWC). Springer, pp. 198–214.

King, I. J., Huang, H. H., 2023. Euler: Detecting network lateral movement via scalable temporal link prediction. ACM Transactions on Privacy and Security 26(3), 1–36.

Koistinen, K., Hellsten, K., Herttuainen, J., Kaski, K. K., 2026. Spatio-temporal attention graph neural network: Explaining causalities with attention. IEEE Access (advance online publication), doi:10.1109/ACCESS.2026.3702352; arXiv:2603.10676.

Kokhlikyan, N., Miglani, V., Martin, M., et al., 2020. Captum: A unified and generic model interpretability library for PyTorch. arXiv preprint arXiv:2009.07896.

Li, H., Chasaki, D., 2024. Heterogeneous GNN with express edges for intrusion detection in cyber-physical systems. In: 2024 International Conference on Computing, Networking and Communications (ICNC). IEEE, pp. 523–529.

Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M., Portmann, M., 2022. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. In: NOMS 2022–2022 IEEE/IFIP Network Operations and Management Symposium. IEEE, pp. 1–9.

Luo, D., Cheng, W., Xu, D., Yu, W., Zong, B., Chen, H., Zhang, X., 2020. Parameterized explainer for graph neural network. Advances in Neural Information Processing Systems 33, 19620–19631.

MITRE, 2024. ATT&CK for Industrial Control Systems. https://attack.mitre.org/matrices/ics/.

Murali, N., Puli, A., Yu, K., Ranganath, R., Batmanghelich, K., 2023. Shortcut learning through the lens of early training dynamics. arXiv preprint arXiv:2302.09344.

Somma, M., 2025. Hybrid temporal differential consistency autoencoder for efficient and sustainable anomaly detection in cyber-physical systems. arXiv preprint arXiv:2504.06320.

Sun, Z., Teixeira, A. M. H., Toor, S., 2024. GNN-IDS: Graph neural network based intrusion detection system. In: Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES). ACM.

Sundararajan, M., Taly, A., Yan, Q., 2017. Axiomatic attribution for deep networks. In: Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 3319–3328.

Sweeten, J., Takiddin, A., Ismail, M., Refaat, S. S., Atat, R., 2023. Cyber-physical GNN-based intrusion detection in smart power grids. In: 2023 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm). IEEE, pp. 1–6.

Taormina, R., Galelli, S., Tippenhauer, N. O., Salomons, E., Ostfeld, A., et al., 2018. Battle of the attack detection algorithms: Disclosing cyber attacks on water distribution networks. Journal of Water Resources Planning and Management 144(8), 04018048.

Tran, D.-H., Park, M., 2024. FN-GNN: A novel graph embedding approach for enhancing graph neural networks in network intrusion detection systems. Applied Sciences 14(16), 6932.

Yan, H., Li, X., Zhang, W., Wang, R., Li, H., Zhao, X., Li, F., Lin, X., 2024. Automatic evasion of machine learning-based network intrusion detection systems. IEEE Transactions on Dependable and Secure Computing 21(1), 153–167.

Yehezkel, A., Elyashiv, E., 2022. A GNN-based approach for detecting network anomalies from small traffic samples. In: 2022 IEEE International Conference on Big Data. IEEE, pp. 6838–6840.

Ying, R., Bourgeois, D., You, J., Zitnik, M., Leskovec, J., 2019. GNNExplainer: Generating explanations for graph neural networks. Advances in Neural Information Processing Systems 32.

Zhong, M., Lin, M., Zhang, C., Xu, Z., 2024. A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. Computers & Security 141, 103821.